Apollo-RTOS: An AGC-Inspired Real-Time Operating System for Embedded Systems

M. Ahsan Al Mahir

Department of Computer Science
University of Oxford

Supervisor: Prof. Alex Rogers

A project report submitted for the degree of Master of Mathematics and Computer Science

Trinity 2025

2599 words (using `texcount`)

Abstract

Embedded systems have evolved dramatically, yet many of the challenges faced by early computing systems remain relevant today. The Apollo Guidance Computer (AGC), despite its limited hardware—a 2 MHz CPU, 4 KB of RAM, and 72 KB of ROM—introduced innovative scheduling and fault-tolerant mechanisms that ensured reliability in mission-critical environments. Inspired by the AGC, this project presents Apollo-RTOS, a real-time

operating system (RTOS) for the Microbit v1, designed to explore the applicability of these historical concepts in modern embedded computing.

Apollo-RTOS implements a hybrid scheduling algorithm, combining cooperative and preemptive multitasking to optimize resource utilization and responsiveness. Additionally, it features a fault-tolerant recovery system, ensuring process integrity even during failures. The system extends the AGC's original design by incorporating modern OS functionalities, including a file system leveraging flash memory and an external FRAM device, inter-process communication through signals, and a shell interface for system interaction. Implemented in modern C++, Apollo-RTOS utilizes RAII, templates, and type deduction to enhance efficiency and maintainability.

This project demonstrates how historical embedded computing techniques can be adapted to contemporary microcontrollers, offering insights into designing resilient, efficient, and resource-aware real-time operating systems for modern applications.

Contents

§1 Introduction

1.1 Background & Motivation

Computing capabilities have grown at an unprecedented rate over the last century. The **Apollo Guidance Computer** (AGC), which played a critical role in the Apollo program, operated with a modest *2 MHz CPU*, *4 KB of RAM*, and *72 KB of ROM*. To maximize its limited resources, the AGC's operating system introduced innovative solutions for *task scheduling*, *error handling*, and *resource-efficient computation*. One of its most groundbreaking features was a hybrid scheduling mechanism that combined cooperative and preemptive multiprogramming, ensuring that critical tasks could execute reliably even in adverse conditions. Additionally, the AGC incorporated a robust recovery mechanism, enabling fault tolerance in the face of system failures.

Today, a compact `micro:bit` device costing less than £20 possesses significantly more computing power than the AGC. Such microcontrollers are now ubiquitous, powering applications ranging from consumer electronics to mission-critical systems. Despite the vast improvements in computational power, many of the challenges that the AGC's operating system addressed remain highly relevant, particularly in constrained embedded environments where real-time performance, reliability, and fault tolerance are essential.

1.2 Project Objectives

The goal of this project is to study the design principles of the AGC's operating system and implement a robust real-time operating system (RTOS) for the `micro:bit` v1, named ***Apollo-RTOS***. This project aims to explore the *applicability of the AGC's novel scheduling techniques and fault-tolerant mechanisms* in modern embedded computing. Additionally, it extends these foundational ideas by incorporating *contemporary operating system concepts* to enhance usability, efficiency, and resilience.

Apollo-RTOS integrates:

- **Hybrid scheduling** that balances cooperative and preemptive multitasking.
- **A fault-tolerant system** capable of handling hardware failures and ensuring process recovery.
- Separate **file systems** implemented on flash memory and an external FRAM device for persistent storage.
- **Inter-process communication** via signals for dynamic event handling.
- **A shell interface** for interacting with the system and managing processes.
- **A modern C++ implementation** leveraging RAII, templates, and type deduction for robustness and efficiency.

By bridging historical embedded computing principles with modern software engineering techniques, this project provides insights into designing efficient, fault-tolerant operating

systems for resource-constrained environments.

§2 The Apollo Guidance Computer

The Apollo Guidance Computer (AGC) was an early digital computer developed in the 1960s to help navigate and control Apollo spacecraft on their missions to the Moon. It provided real-time guidance and control, making it essential for the success of the Apollo program.

The AGC ran at a speed of 2.048 MHz, which translated to about 40,000 instructions per second. The AGC had a 16-bit `word` length, 15 bits for data, and one parity bit. It had 2 kilowords or 4KB of erasable magnetic-core memory (RAM) along with 36 kilowords or 72KB of fixed core rope memory (ROM) for storing software and mission data. Astronauts interacted with the AGC using the Display and Keyboard (DSKY), a 21-digit display with a 19-button keyboard that allowed them to input commands and receive information. This interface was critical for making manual adjustments and monitoring the system during missions.

Due to these constraints, the operating system, called the Executive, had to efficiently manage resources. It ensured that important tasks were prioritized over less critical ones and that key functions could continue even if the system experienced a reset due to an error.

The Executive was written in AGC assembly, where each instruction was 16 bits long. Because the instruction set was small, the system also included a sophisticated interpreter that acted as a virtual machine, providing more advanced pseudo-instructions beyond the basic AGC instructions.

Figure 0.1: Margaret Hamilton standing next to listings of the software she and her MIT team produced for the Apollo Project.

The details of the AGC Executive explained below is taken from O'Brien [1].

2.1 The Executive

The central component of the AGC software was the Executive, which ensured fair allocation of system resources, such as CPU time and dynamic memory, to processes and tasks. To seamlessly perform real-time control and navigation tasks with extreme reliability and efficiency, the Executive implemented a **priority-based cooperative scheduling** algorithm, described below:

- The highest-priority process has exclusive access to the CPU.
- Lower-priority processes remain idle until the highest-priority process:
  - lowers its priority, or
  - goes to `sleep` while waiting for an event.
- If a new process with a higher priority is scheduled while another is running:
  - the new process waits until the current process lowers its priority or goes to `sleep`,
  - then the new process starts executing.

Core Sets Information about running processes was stored in a table called Core Sets, which functioned similarly to modern process descriptors. Each Core Set had a structure similar to:

```
struct CORE_SET {
    word priority;
    word program_counter;
    word bbank_reg; // area in unswitched memory where the code is
    word vac_ptr;   // pointer to extra memory

    word mpac[7];   // multi-purpose accumulators
    word mpac_mode;
};
```

Each active process occupied one Core Set. The system provided:

- 6 Core Sets in the Command Module, and
- 7 Core Sets in the Lunar Module.

In either case, Core Set 0 always contained the currently running process. A context switch was essentially performed by swapping the current Core Set with Core Set 0.

Addressable Memory The 7 `mpac` words served two purposes. In normal mode, they provided 7 words of freely usable memory. In interpreter mode, they functioned as accumulator registers, storing values for calculations performed by the interpreter. If a process required more memory, it could request 42 additional words from the **Vector Accumulator** (VAC) area.

Starting a New Process To start a new process, an empty Core Set had to be located, followed by a VAC area if needed. After populating the Core Set, the Executive determined if the new process had a higher priority.

A special register, `NEWJOB`, was used for this purpose:

- If `NEWJOB` is 0, the current process has the highest priority.
- If `NEWJOB` is non-zero (say `d`), the process represented by Core Set `d` has the highest priority.

When a new process was created, its priority was compared to that of the currently running process, and `NEWJOB` was updated accordingly.

Process Switching A context switch only occurred when the current process voluntarily relinquished control. The process was required to check `NEWJOB` at least every 20ms to detect higher-priority processes. If `NEWJOB` was not checked for over 640ms, it was assumed that the process had stalled, triggering a hardware restart.

To switch processes, the Executive stored the current context in Core Set 0 and swapped it with the Core Set indexed by `NEWJOB`. Since the program counter was restored directly from the Core Set, execution of the switched-in process resumed automatically from its last instruction.

If the current process was still the highest-priority process but needed to go to sleep or lower its priority, the system scanned the Core Set table to identify the next highest-priority process and performed a switch accordingly.

Sleep and Wakeup A process could voluntarily relinquish execution while waiting for an external event by going to sleep. The `JOBSLEEP` instruction marked a process as sleeping by negating its priority. The next highest-priority process was then scheduled.

When the awaited event occurred, another process could wake the sleeping process using the `JOBWAKE` instruction. This reversed the priority negation, making the process runnable again. `JOBWAKE` then checked `NEWJOB` to determine if the woken process should run immediately.

A process could also call `DELAYJOB` to sleep for a fixed duration. This scheduled an "alarm" task via the **waitlist**, described in the next section.

2.1.1 The Waitlist

The core processes in the Executive were all cooperative, meaning that even the highest-priority process would not preempt the currently running process until it voluntarily relinquished execution. The waitlist introduced preemptive scheduling capabilities to the AGC, allowing tasks to be scheduled with strict deadlines. Every 10ms, a timer interrupt was triggered to execute any waitlist task that was due.

In essence, the waitlist was a ***sorted linked list*** of tasks ordered by increasing deadlines. When a new waitlist task t was scheduled with a particular deadline, it was inserted into the list such that:

- any task t_0 with an earlier deadline appeared before t, and

- any task t_1 with a later deadline appeared after t.

Every 10ms, the timer interrupt checked whether the first task in the list had reached its deadline. If so, the task was executed and then removed from the list. Since waitlist tasks were executed within an interrupt handler, they had to be short-lived—typically around 5ms or 150–200 instructions.

The waitlist implementation in Apollo-RTOS closely follows that of the AGC. Implementation details will be discussed later.

2.2 Restart Mechanism

Despite its limited resources, the AGC needed to be highly reliable, even in the presence of errors. Modern CPU and OS designs dedicate significant resources to detecting and recovering from faults. However, the AGC lacked the hardware to support sophisticated error handling. Furthermore, without a memory protection unit, it was nearly impossible to reliably detect errors caused by memory corruption. As a result, the AGC adopted a radical approach:

Just restart everything.

However, restarting at an arbitrary point in time required careful handling, as it could terminate critical tasks mid-execution.

To address this, critical tasks and processes could define recovery behavior in case of a restart. Each program belonged to one of six restart groups. A process could use the `PHASCHNG` instruction to set up an entry in the restart phase table. Each group could recover only one process at a time. Based on its needs, a process could configure its group's recovery procedure to one of the following four modes:

- **Inhibit group restart:** No processes in this group are restarted.
- **Restart last display:** Restore and display the last known contents before the restart.
- **Execute one restart routine:** Restart the process from the beginning with minimal cleanup.
- **Execute two restart routines:** Schedule a task to clean up data and re-enter the process from a designated point.

To conserve memory, the AGC stored recovery information in a compressed format within the phase tables.

During a restart:

- All active jobs were canceled, and all tasks in the waitlist queue were deleted.
- Interrupts were disabled, and all tables were cleared and reinitialized.
- The integrity of the restart phase table was verified.
- Restart processing was carried out sequentially for each active phase in groups 6 to 1.

These mechanisms ensured that the AGC could gracefully recover from many types of fail-

ures—an essential requirement for spacecraft operations, where computer failures could have catastrophic consequences.

2.3 Interpreter

The Apollo Guidance Computer (AGC) featured a custom virtual machine known as the **Interpreter**, designed to extend its limited instruction set. This was necessary because the AGC's native instruction set was highly restricted, containing only 11 opcodes in AGC Block II. The Interpreter enabled more complex operations while conserving memory and improving execution efficiency for key tasks.

The Interpreter functioned as a *bytecode interpreter*, running on top of the AGC's native instruction set. It allowed the execution of *pseudo-instructions*, which were more powerful than basic machine instructions. These pseudo-instructions were stored as compact *6-bit opcodes*, maximizing memory efficiency.

The AGC Interpreter provided an extended set of *arithmetic and vector operations*, primarily aiding navigation and guidance computations. Key capabilities included:

- **Arithmetic Operations**: Fixed, single, and double-precision floating-point arithmetic, as well as `sqrt` calculations.
- **Vector and Matrix Operations**: Dot products, cross products, and matrix transformations.
- **Trigonometric Functions**: Sine, cosine, and related functions for celestial navigation.
- **Data Movement and Manipulation**: Indexed memory access (not available in native AGC instructions), indirect addressing, and array operations.
- **Flow Control**: Conditional branching and subroutine-like operations for reusable code execution.

The AGC's native instruction set prioritized speed but lacked flexibility for advanced navigation tasks. The Interpreter enabled *higher-level mathematical operations* to be executed compactly and efficiently.

2.4 Display and Keyboard

The AGC featured a unique human-machine interface known as the *Display and Keyboard (DSKY)*, allowing astronauts to communicate with the computer using an intuitive *verb-noun paradigm*. With its green *electroluminescent displays*, *19-key input panel*, and *status indicators*, the DSKY played a crucial role in both the Command Module and Lunar Module cockpits. Despite its simple appearance, it incorporated sophisticated software routines, enabling astronauts to *monitor spacecraft systems*, *compute trajectories*, *and execute maneuvers with precision*.

The *verb-noun paradigm* structured commands similarly to natural language, where astronauts first selected a *verb* (the action) followed by a *noun* (the object of that action). Using

two-digit numeric codes for both components, astronauts could issue commands efficiently. For example, the command *"Verb 06 Noun 36"* instructed the AGC to *display the current time*. This system provided a consistent interface for a range of operations, from *simple data retrieval to complex trajectory calculations*.

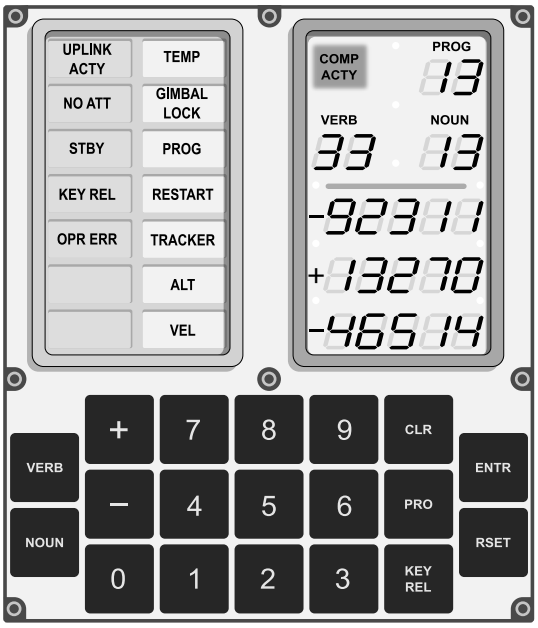

Figure 0.2: DSKY interface diagram by Oona Räisänen

§3 Inspiration behind Apollo-RTOS

The real-time operating system developed in this project, *Apollo-RTOS*, draws inspiration from both the Apollo Guidance Computer (AGC) operating system and the modern Linux OS. While the scheduler and error handling concepts are derived from the AGC, the naming conventions, user interface, and implementation are influenced by Linux.

The AGC OS was designed to meet specific mission requirements, allowing for a simplified and optimized design. In contrast, Linux is a general-purpose OS, designed to support applications with vastly different needs, making its architecture more complex.

The AGC operated within a well-defined hardware and software environment, where all components could be trusted. As a result, isolating the hardware, OS, and application code was unnecessary. Additionally, the AGC lacked memory protection features, making kernel/user space isolation, a fundamental aspect of general-purpose operating systems like Linux, impossible.

In Apollo-RTOS, we adopted the same high-level design principles as the AGC. Our goal was to implement its key innovations in a modern context while ensuring the OS could function efficiently in a resource-constrained environment. To achieve this, we made the following design choices:

- **Hardware Selection:** The `micro:bit` v1 was chosen as the primary target platform. Its Cortex-M0 CPU is one of the smallest in the ARM family and closely resembles the AGC hardware. Additionally, it includes pre-installed peripheral devices, which serve as useful test cases for OS functionality.
- **No Kernel/User Space Isolation:** In microcontroller programming, the entire application is compiled into a single binary, meaning the kernel and user space are inherently combined. Since the entire codebase has full access to the kernel internals, enforcing strict isolation would add unnecessary complexity.
- **Monolithic Kernel Design:** Inspired by Linux, Apollo-RTOS follows a monolithic kernel architecture due to its simpler implementation and better execution efficiency in a constrained environment.
- **Library Calls Instead of System Calls**: Traditional system calls are beneficial only when user and kernel space are strictly separated. Since Apollo-RTOS does not enforce such isolation, we opted for direct library function calls, significantly simplifying implementation.
- **Shared Memory for Inter-Process Communication (IPC):** While shared memory introduces risks like race conditions and synchronization issues, it remains the fastest and simplest IPC method, making it the preferred choice for Apollo-RTOS.

Additionally, we drew inspiration from the `microbian` OS for the `micro:bit`, as developed by Spivey [2]. Specifically, we referenced its hardware description headers, startup code, and interrupt handler implementations in certain parts of our design.

§4 Implementation Architecture and Design

(i) Overall system architecture

(ii) Design principles and approach

(iii) Adaptations for micro:bit constraints

(iv) High-level organization and module relationships

§5 Implementation Details

5.1 Core Executive System

(i) Task scheduling and prioritization
(ii) Interrupt handling
(iii) Time management
(iv) Original AGC features preserved vs. modified

5.2 Memory Management

(i) Memory organization and allocation
(ii) FRAM extension implementation
(iii) Comparison with AGC's memory management

5.3 I/O Subsystem

(i) Interface with micro:bit peripherals
(ii) Mapping AGC I/O concepts to micro:bit
(iii) Novel I/O approaches

5.4 Key Algorithms and Procedures

(i) Implementation of critical AGC algorithms
(ii) Performance optimizations
(iii) Code examples of significant components

§6 Evaluation and Testing

(i) Testing methodology on micro:bit
(ii) Performance metrics
(iii) Reliability considerations
(iv) Comparison with original AGC capabilities
(v) Limitations and trade-offs

§7 Future Work

(i) Potential improvements and extensions

(ii) Unresolved challenges

(iii) Research or application possibilities

§8 Conclusion

(i) Summary of achievements

(ii) Reflections on the AGC's enduring design principles

(iii) Broader implications for OS design

Reference

[1] Frank O'Brien. ***The Apollo Guidance Computer: Architecture and Operation***. 01 2010. ISBN 978-1-4419-0876-6. doi: 10.1007/978-1-4419-0877-3.

[2] Mike Spivey. micro:bian. `https://github.com/Spivoxity/microbian`, 2021.

§A Appendices

(i) Code snippets of key algorithms

(ii) Detailed specifications

(iii) Test results